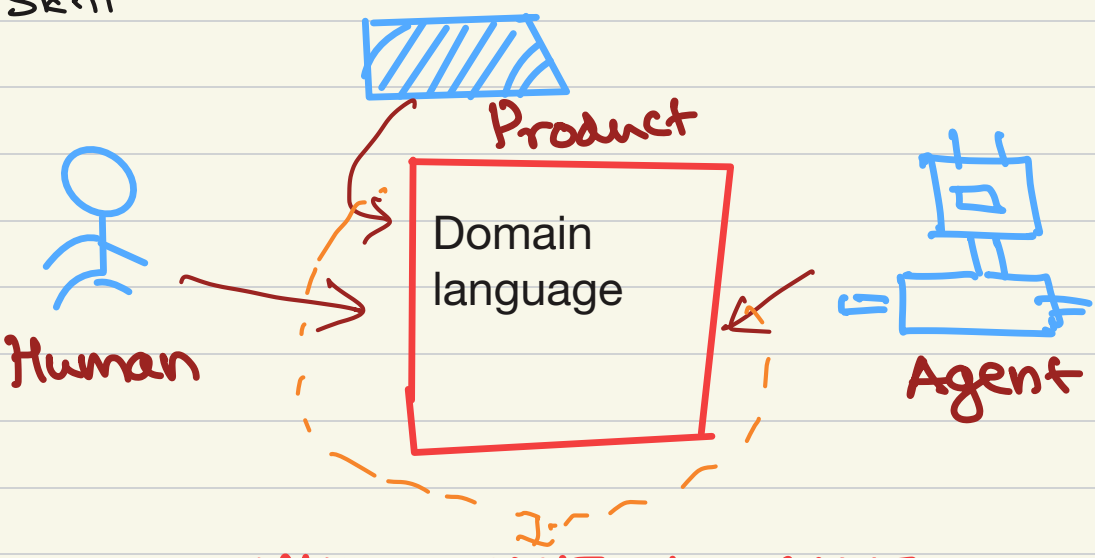
Domain Language

- Need to have a common language that a human or agent can understand.
- We can name it as "Glossary.md"
- Need to have "domain modelling" skill



- inspired from "Domain driven design"

EXPLORE

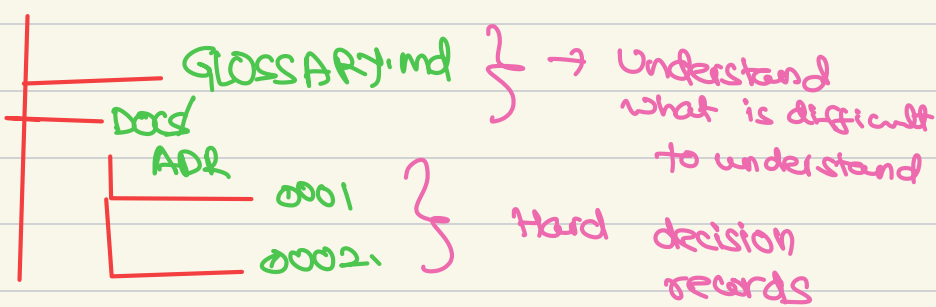
Explore the internals of the book and create skill

GRILL WITH DOCS

- Relentlessly ask AI to grill so that both can be on the same page.
- Before that we need to have same glossary message.
- e.g. "Refund" or "Return" or "cancel"

DOMAIN MODELLING

- Need to keep ADR (Architecture Decision Records)
- Offer ADR
  - Hard to reverse
  - Based on tradeoff decision
  -



WRITING AN ADR

- When grilling we update GLOSSARY and then we decide some decision that should be recorded.
- The decision could be tech decision or business decision.
- Think about any business decision that is being made. by using another skill called /grill me with docs.
- Once Glossary is updated ↑ skills really helps. It asks right questions
- This exercise have to be done manually

TRY THIS

- Need to try this in each repository
  ↓
  Question → Do we create GLOSSARY for one repo or multiple repo.

Pruning The Glossary

- Over period of time the glossary keep getting bloated.
  - This needs to be cleaned on regular basis.
  - "/grill-with-docs" to discuss and prune unused glossary
  - Tip 1 - Implementation details should not be added
    e.g. "Refund" db details

The reason for this is because the implementation can change and code should be source of truth.

Tip2 Any coding language details for React, Java etc.

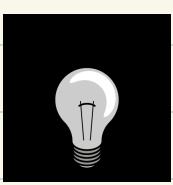
Its not needed as we don't need things here.

Pruning ADR

- Similarly we need to do cleaning of ADR
- Remember ADR's are
  1. Hard to reverse
  2. Surprising
  3. Based on tradeoff example

Based on DDD Book

- Use same skill - /grill-with-docs to prune the ADR



Can we make this documentation update tasks e.g. Glossary or ADR as loop engineering

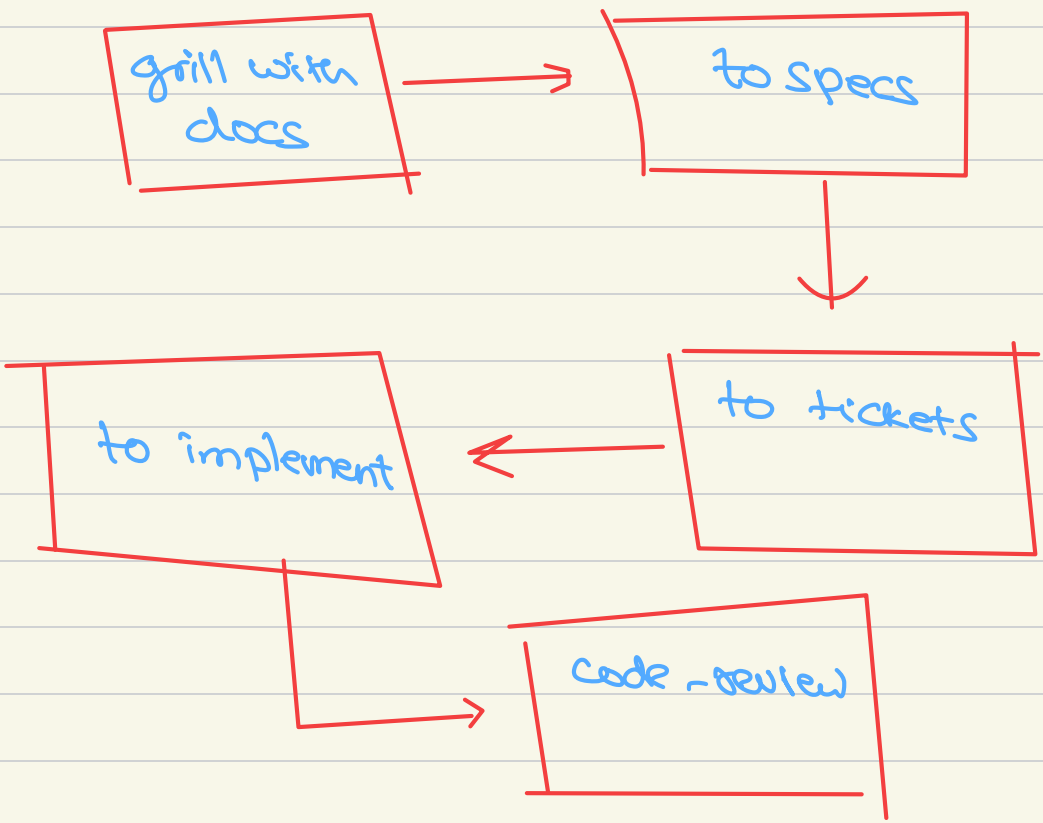
Every ~~PR~~ merge

too much change !!

Regular async job

SUMMARY OF CH1

- In DDD, Company wage single language to talk to. Try to keep that language.
- For this keep the same bounded Context. use / domain-modelling skills.
-



- What to choose for good words.

  ⇓

  The language should be used what users they might be using. To make this easy to use.
- go slowly, → create only entries that we need and grow as you need.
-

Bounded Context

TO READ

Domain Driven Design

Ubiquitous language